

Detección de mensajes SMS spam mediante fine-tuning de FLAN-T5-small: comparación cuantitativa frente al modelo base zero-shot

Arianna Faustos · Ronny Oswaldo Guamán · Erick Cabrera · Carlos Andrés Anchundia Toala · Jair Jander Iloor Holguín · Fernando Baque

Redes Neuronales y Aprendizaje Profundo — Proyecto de fin de parcial (Segundo Parcial)

Repositorio: [Repositorio - Github](#) · Fecha: 20/07/2026

Resumen— Este trabajo adapta el modelo pre-entrenado FLAN-T5-small (77 M de parámetros) a la clasificación de mensajes SMS como legítimos (ham) o no deseados (spam), siguiendo la estrategia texto-a-texto propuesta por Labonne y Moran en Spam-T5 [1]. Sobre el dataset público SMS Spam Collection (5 160 mensajes tras la limpieza) se estableció primero una línea base evaluando el modelo sin entrenar mediante zero-shot prompting, y después se aplicó fine-tuning completo durante tres épocas. Con idénticas métricas sobre el mismo conjunto de prueba (774 mensajes), el F1 de la clase spam pasó de 0.229 a 0.990 (+0.761) y la exactitud de 0.164 a 0.997, con solo dos errores residuales. El análisis muestra que el modelo base no discriminaba las clases, sino que respondía “spam” de forma casi sistemática, y que la adaptación con 3 612 ejemplos etiquetados corrige ese sesgo. Los resultados superan incluso al $F1 = 0.95$ reportado por el paper con FLAN-T5-base, diferencia que se explica por el régimen de entrenamiento completo frente al enfoque few-shot del estudio original. Se discuten los errores restantes, se propone un mecanismo de abstención por confianza y se describe el proceso de despliegue del modelo como servicio real de filtrado.

Palabras clave— detección de spam, fine-tuning, transfer learning, FLAN-T5, modelos de lenguaje, clasificación de texto, few-shot learning.

1. Introducción

1.1 Descripción del problema

La detección de spam es un problema clásico de clasificación binaria de texto que, pese a su larga trayectoria, sigue planteando retos reales: el desbalance entre mensajes legítimos y fraudulentos, el cambio constante del vocabulario utilizado por los emisores (concept drift) y el carácter adversarial del entorno, en el que los atacantes modifican deliberadamente sus mensajes para evadir los filtros [1]. Los métodos tradicionales (Naive Bayes, SVM, árboles potenciados) requieren volúmenes considerables de datos etiquetados y reentrenamientos frecuentes. Los modelos de lenguaje pre-entrenados ofrecen una alternativa: transferir el conocimiento lingüístico general adquirido durante el pre-entrenamiento y adaptarlo a la tarea con un ajuste relativamente corto.

Este proyecto responde a una pregunta concreta y medible: *¿en qué medida el fine-tuning de FLAN-T5-small sobre el dataset SMS Spam Collection mejora la clasificación de mensajes como ham o spam, en comparación con el mismo modelo evaluado en zero-shot?*

1.2 Trabajo de referencia

El paper de referencia es *Spam-T5: Benchmarking Large Language Models for Few-Shot Email Spam Detection* [1]. Sus autores comparan seis modelos tradicionales de aprendizaje automático (Naive Bayes, regresión logística, KNN, SVM, XGBoost y LightGBM) con tres modelos de lenguaje de familias distintas (RoBERTa, SetFit y FLAN-T5) sobre cuatro datasets públicos: Ling-Spam, SMS Spam Collection, SpamAssassin y Enron. La evaluación cubre tanto el entrenamiento con el conjunto completo como escenarios few-shot con 4, 8, 16, 32, 64, 128 y 256 ejemplos.

Su principal aporte es **Spam-T5**, una versión de FLAN-T5 ajustada para la tarea: a cada mensaje se le antepone la instrucción “`classify as ham or spam:`” y el modelo se entrena para generar directamente una de las dos etiquetas. Spam-T5 obtiene el mejor desempeño global del estudio, con un F1 promedio de 0.9742 en entrenamiento completo y de 0.7498 considerando todos los tamaños de entrenamiento, destacando especialmente en los escenarios con muy pocos ejemplos. Los autores construyeron Spam-T5 sobre FLAN-T5-base (250 M de parámetros) y descartaron la versión small por sus limitadas capacidades de generalización, un punto que este proyecto retoma en la discusión (sección 5.4).

1.3 Objetivos e hipótesis

Objetivo general. Adaptar FLAN-T5-small a la detección de spam en SMS mediante fine-tuning completo y cuantificar la mejora respecto al modelo base sin adaptar.

Objetivos específicos. (i) Preparar y analizar el dataset SMS Spam Collection; (ii) establecer la línea base evaluando el modelo en zero-shot sobre el conjunto de prueba; (iii) aplicar fine-tuning texto-a-texto siguiendo la estrategia de [1]; (iv) comparar ambos modelos con las mismas métricas sobre el mismo split de prueba; (v) analizar críticamente los resultados, los errores y las limitaciones del enfoque.

Hipótesis. El modelo fine-tuneado superará de forma sustancial al modelo base zero-shot en F1 de la clase spam y en F1 macro, dado que el ajuste con ejemplos etiquetados corrige la incapacidad del modelo base para discriminar las clases a partir únicamente de la instrucción del prompt.

1.4 Justificación

Se eligió este problema y este paper porque la adaptación de un modelo pre-entrenado es la contribución central del trabajo de referencia, el dataset SMS Spam Collection es uno de los cuatro que el propio paper evalúa (lo que permite una comparación directa de resultados), y la escala del modelo small hace viable el fine-tuning completo en Google Colab con recursos limitados, garantizando la reproducibilidad del experimento por parte de cualquier integrante del grupo.

2. Dataset y análisis exploratorio

2.1 Descripción, adquisición y limpieza

Se utilizó el corpus público SMS Spam Collection [5], disponible en Hugging Face con el identificador `ucirvine/sms_spam` y descargado con código dentro del notebook, en cumplimiento de las normas de entrega. El conjunto original contiene 5 574 mensajes en inglés con dos campos: el texto del SMS y la etiqueta binaria (0 = ham, 1 = spam). La limpieza eliminó espacios externos, textos vacíos, valores nulos y 414 mensajes duplicados, dejando 5 160 registros: 4 518 ham (87.56 %) y 642 spam (12.44 %). Se verificó además que no existieran mensajes idénticos con etiquetas contradictorias.

2.2 Particiones

El dataset limpio se dividió de forma estratificada en tres particiones (70/15/15) con semilla fija (42), de modo que las tres conservan la proporción de clases del conjunto completo (Tabla 1). El conjunto de prueba se mantuvo intacto y fue utilizado exactamente igual por ambos modelos.

Tabla 1. Composición de las particiones estratificadas.

Partición	Total	Ham	Spam	% spam
Entrenamiento	3 612	3 163	449	12.43
Validación	774	677	97	12.53
Prueba	774	678	96	12.40

2.3 Hallazgos del análisis exploratorio

El análisis exploratorio confirmó el desbalance característico del problema (Figura 1, izquierda), que obliga a interpretar las métricas con cuidado: un clasificador trivial que responda siempre “ham” alcanzaría 87.6 % de exactitud sin detectar un solo mensaje spam. El estudio de longitudes reveló además una diferencia estructural entre clases: los mensajes spam tienden a ser más largos y homogéneos que los legítimos (Figura 1, derecha), consistente con su naturaleza promocional (premios, instrucciones de contacto, enlaces). Esta señal, sin embargo, no basta por sí sola para clasificar, lo que motiva el uso de un modelo de lenguaje capaz de capturar patrones léxicos y de estilo.

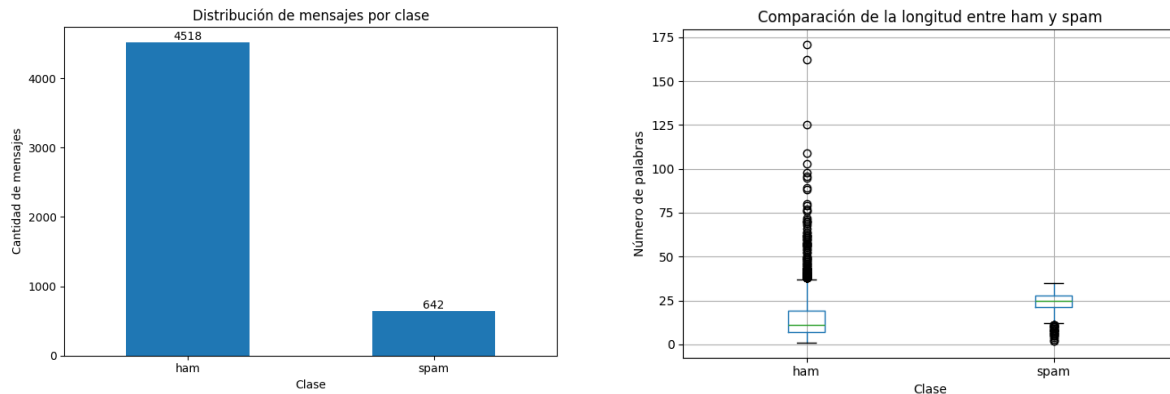


Figura 1. Distribución de clases tras la limpieza (izquierda) y comparación de la longitud en palabras entre ham y spam (derecha).

3. Metodología e implementación

3.1 Flujo de trabajo experimental

La Figura 2 resume el flujo completo del experimento. El requisito central del proyecto es la comparación cuantitativa antes/después: el mismo conjunto de prueba, las mismas métricas y el mismo procedimiento de normalización de respuestas se aplican tanto al modelo base sin adaptar como al modelo fine-tuneado, de modo que cualquier diferencia observada sea atribuible exclusivamente a la adaptación.

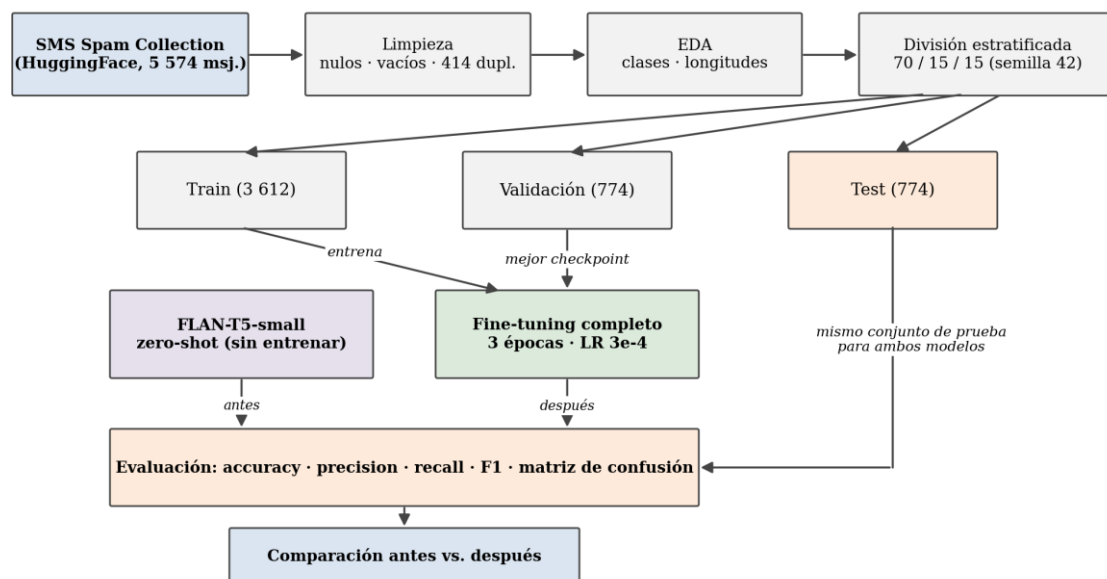


Figura 2. Flujo de trabajo experimental: preparación de datos, línea base zero-shot, fine-tuning y evaluación comparativa sobre el mismo conjunto de prueba.

3.2 Formulación texto-a-texto

FLAN-T5 [2] es un transformer encoder-decoder [3], [4] afinado con instrucciones sobre más de 1 800 tareas. En lugar de añadir una cabeza de clasificación, la tarea se formula como generación de texto (Figura 3): cada mensaje se envuelve en el prompt “Classify the following SMS message as spam or ham: ... Answer only spam or ham:”, el modelo genera una respuesta corta y una función de normalización la reduce a una de tres categorías: spam, ham o invalid (cuando el texto generado no corresponde de forma inequívoca a ninguna etiqueta). Esta tercera categoría se contabiliza explícitamente y las métricas se implementaron de forma robusta a su presencia, de modo que el notebook se ejecuta de inicio a fin aun si el modelo produjera respuestas fuera del vocabulario esperado.

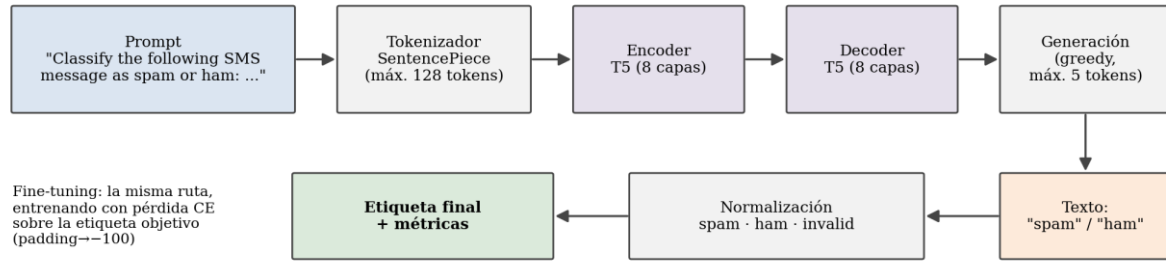


Figura 3. Flujo de inferencia texto-a-texto con FLAN-T5-small (77 M de parámetros, 8 capas de encoder y 8 de decoder). En el fine-tuning se recorre la misma ruta optimizando la pérdida de entropía cruzada sobre la etiqueta objetivo.

3.3 Línea base: evaluación zero-shot

Se cargó google/flan-t5-small sin ningún entrenamiento adicional y se clasificaron los 774 mensajes de prueba mediante inferencia por lotes (tamaño 32, generación greedy, máximo 5 tokens nuevos). La inferencia completa tomó 5.67 s en una GPU T4 (≈ 7.3 ms por mensaje) y no produjo ninguna respuesta inválida. Estas predicciones constituyen el “antes” obligatorio de la comparación.

3.4 Estrategia de adaptación: fine-tuning completo

Se aplicó *full fine-tuning* (ajuste de todos los parámetros) sobre una instancia independiente del modelo, conservando intacto el modelo base como referencia. Cada ejemplo de entrenamiento es el par (prompt, etiqueta): la entrada se tokeniza con SentencePiece a un máximo de 128 tokens y la salida a 4, sustituyendo los tokens de padding de la etiqueta por -100 para que la pérdida de entropía cruzada los ignore. El entrenamiento usó el Seq2SeqTrainer de la librería transformers [6] con los hiperparámetros de la Tabla 2, seleccionando el mejor checkpoint por pérdida de validación.

Tabla 2. Hiperparámetros del fine-tuning y su justificación.

Hiperparámetro	Valor	Justificación
Épocas	3	La pérdida de validación se estabiliza temprano; más épocas arriesgan sobreajuste
Learning rate	3e-4	Orden de magnitud habitual para fine-tuning de la familia T5
Batch (train/eval)	16 / 32	Máximo estable en la GPU T4 de Colab para esta longitud
Weight decay	0.01	Regularización estándar de AdamW
Long. máx. (entrada/salida)	128 / 4	SMS cortos; la salida es una etiqueta más el token de fin
Precisión fp16	Desactivada	T5 se pre-entrenó en bfloat16; float16 produce pérdidas NaN
Selección de modelo	Mín. eval_loss	Mejor checkpoint por época; semilla 42 para reproducibilidad

Dos hallazgos prácticos merecen registro. Primero, al intentar acelerar el entrenamiento con precisión mixta (fp16=True), la pérdida y los gradientes colapsaron a NaN desde la primera época: el rango dinámico de float16 es insuficiente para las activaciones de T5, pre-entrenado en bfloat16, por lo que se

desactivó por completo. Segundo, el argumento `tokenizer=` del entrenador fue renombrado a `processing_class=` en versiones recientes de la librería, error de compatibilidad que hubo que diagnosticar durante el desarrollo.

4. Resultados: antes y después de la adaptación

4.1 Dinámica del entrenamiento

El entrenamiento completo (678 pasos, 3 épocas) tomó 167.6 s (≈ 2.8 min) en GPU T4. La Tabla 3 y la Figura 4 muestran la evolución de las pérdidas: la pérdida de entrenamiento descende de forma monótona, mientras que la de validación alcanza su mínimo en la segunda época (0.0181) y repunta levemente en la tercera, indicio incipiente de sobreajuste. El mecanismo de selección por pérdida de validación recuperó automáticamente el checkpoint de la época 2 como modelo final.

Tabla 3. Pérdidas por época durante el fine-tuning.

Época	Pérdida de entrenamiento	Pérdida de validación
1	0.0410	0.0256
2	0.0066	0.0181 (mínimo)
3	0.0020	0.0204

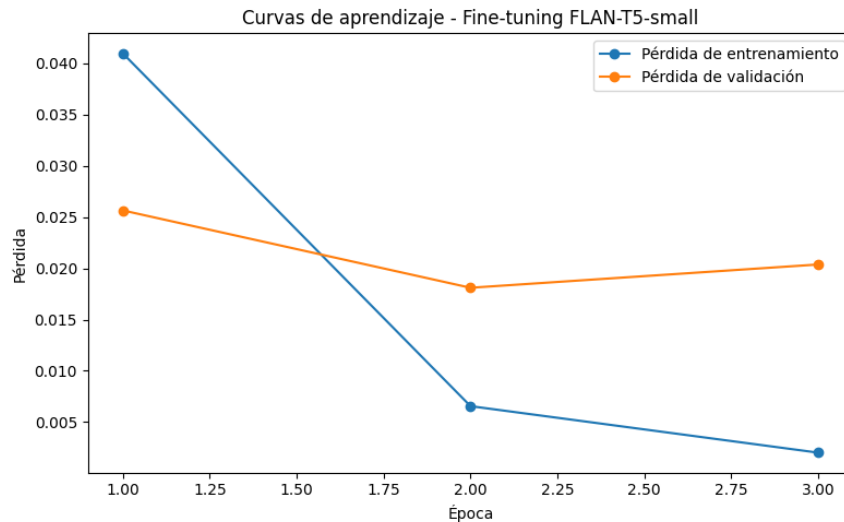


Figura 4. Curvas de aprendizaje del fine-tuning de FLAN-T5-small.

4.2 Comparación cuantitativa

La Tabla 4 presenta la comparación central del proyecto: las mismas cinco métricas, calculadas sobre el mismo conjunto de prueba, para el modelo base y el modelo adaptado. La Figura 5 las representa gráficamente. Ninguno de los dos modelos produjo respuestas inválidas.

Tabla 4. Métricas sobre el conjunto de prueba (774 mensajes): modelo base zero-shot frente a modelo fine-tuneado.

Métrica	Modelo base	Fine-tuneado	Diferencia
Accuracy	0.1641	0.9974	+0.8333
Precision (spam)	0.1292	0.9896	+0.8604
Recall (spam)	1.0000	0.9896	-0.0104
F1 (spam)	0.2288	0.9896	+0.7607
F1 macro	0.1581	0.9941	+0.8359
Respuestas inválidas	0	0	0
Tiempo de inferencia (s)	5.67	3.11	-2.56

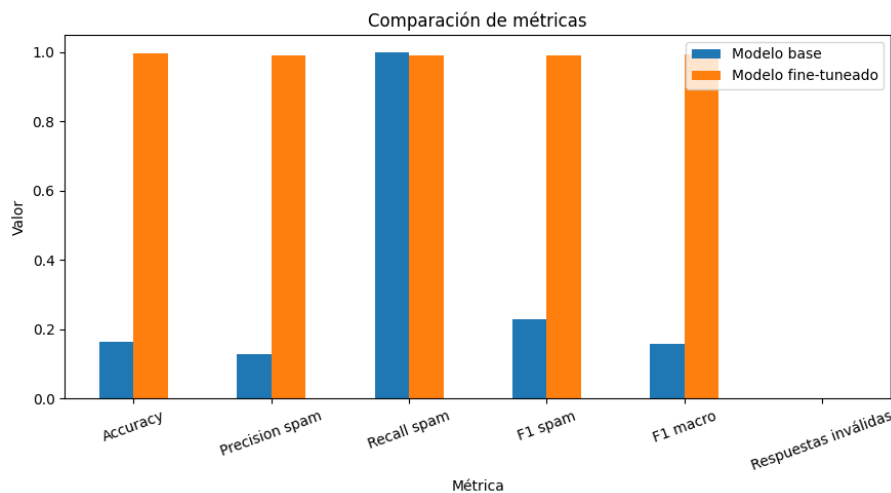


Figura 5. Comparación gráfica de las métricas del modelo base y del modelo fine-tuneado.

4.3 Matrices de confusión

Las matrices de confusión (Figura 6) hacen visible el cambio de comportamiento. El modelo base clasificó como spam 647 de los 678 mensajes legítimos; el modelo adaptado solo confundió un mensaje de cada clase. De los 647 mensajes que el modelo base clasificó incorrectamente, el fine-tuning corrigió 646.

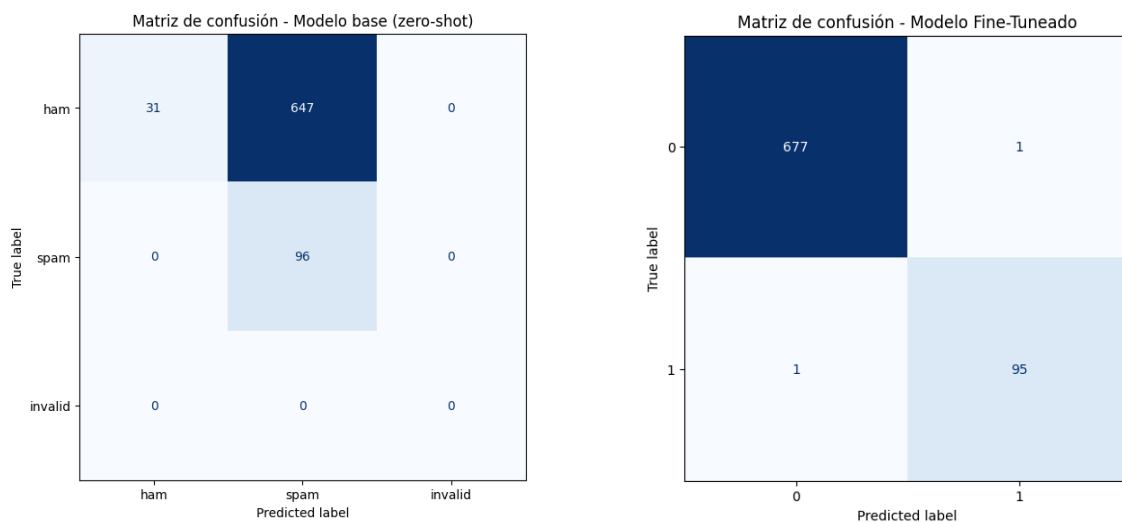


Figura 6. Matrices de confusión del modelo base zero-shot (izquierda) y del modelo fine-tuneado (derecha) sobre el mismo conjunto de prueba.

4.4 Análisis cualitativo de los errores residuales

Falso negativo. El mensaje “*TheMob>Yo yo yo-Here comes a new selection of hot downloads for our members to get for FREE! Just click & open the next link sent to ur fone...*” (real: spam) fue clasificado como ham por el modelo adaptado. Curiosamente, el modelo base lo acertó, aunque por la razón equivocada: su tendencia general a responder “spam”. El modelo fine-tuneado cometió aquí un error genuino de generalización ante un formato promocional (prefijo de remitente, puntuación irregular, jerga) poco representado en el entrenamiento.

Falso positivo. El mensaje “*Total video converter free download type this in google search:)*” (real: ham) fue clasificado como spam por ambos modelos. Contiene vocabulario fuertemente asociado al spam (“free download”) dentro de una recomendación legítima entre conocidos; es un caso ambiguo incluso para un lector humano sin más contexto de la conversación.

5. Análisis crítico

5.1 Significancia de la mejora

La mejora es sustancial en todas las métricas relevantes. La única que desciende, el recall de spam (de 1.000 a 0.990), lo hace porque el valor inicial era un artefacto: un modelo que responde “spam” de forma casi sistemática detecta por construcción todo el spam real. Una ganancia de 0.76 puntos de F1 y de 0.84 de F1 macro es demasiado grande para atribuirla al azar o a la variabilidad de una sola partición: el modelo pasó de no discriminar las clases a clasificar correctamente 772 de 774 mensajes.

5.2 Por qué funciona la adaptación

La clave está en el diagnóstico del modelo base: recall de 1.00 con precisión de 0.13 significa que en zero-shot FLAN-T5-small no analizaba el contenido, sino que la instrucción del prompt sesgaba la generación hacia la etiqueta “spam”. El fine-tuning sustituye esa dependencia exclusiva del prompt por evidencia: 3 612 ejemplos reales con los que el modelo ajusta sus parámetros para reconocer los patrones que separan las clases (números de contacto, términos como “free”, “win” o “claim”, mayúsculas excesivas, tono de urgencia). Este resultado ilustra también una lección metodológica: una métrica aislada (el recall perfecto) puede ocultar un sesgo severo; en problemas desbalanceados es imprescindible leer varias métricas en conjunto.

5.3 Limitaciones

- Los resultados provienen de una sola partición y un solo entrenamiento; con 774 mensajes de prueba y 2 errores, el intervalo de confianza de las métricas es amplio y no se estimó su variabilidad con validación cruzada ni múltiples semillas.
- Se evaluó un solo dominio (SMS); el paper cubre cuatro canales de comunicación, por lo que no puede afirmarse que el desempeño se generalice, por ejemplo, al correo corporativo.
- No se evaluó el escenario few-shot, foco central del paper y contexto donde el tamaño del modelo más debería importar.
- No se midió robustez ante concept drift ni adversarial drift: el modelo solo se probó sobre mensajes de la misma distribución temporal que los de entrenamiento.
- Se ajustaron todos los parámetros (full fine-tuning) en lugar de una técnica eficiente como LoRA; con 77 M de parámetros no fue una limitación práctica, pero lo sería al escalar.

5.4 Contraste con el paper de referencia

Para SMS Spam Collection en entrenamiento completo, el paper reporta $F1 = 0.95$ (precisión 0.97, recall 0.94) con Spam-T5 sobre FLAN-T5-base [1]. Este proyecto, usando la versión small que los autores descartaron, obtuvo $F1 = 0.99$ sobre el mismo dataset. La aparente contradicción se resuelve por el régimen de entrenamiento: la desventaja de un modelo pequeño se manifiesta cuando debe generalizar a partir de 4-256 ejemplos, el escenario que el paper prioriza; con $\approx 3\,600$ ejemplos etiquetados, incluso 77 M de parámetros bastan para aprender la tarea. La conclusión central del paper (los modelos de lenguaje superan ampliamente a los métodos tradicionales y al modelo sin adaptar) se confirma; su decisión de descartar la versión small resulta válida para few-shot, pero no se extiende al entrenamiento completo.

5.5 Propuesta de mejora

A partir de los dos errores residuales se proponen tres líneas concretas: (i) aumentar la representación de spam con formato promocional atípico mediante aumento de datos o ejemplos de Ling-Spam y SpamAssassin, para reducir falsos negativos como el observado; (ii) incorporar un mecanismo de confianza y abstención que derive a revisión manual las predicciones de baja probabilidad, como el falso positivo ambiguo; y (iii) replicar el protocolo few-shot del paper ($k = 4$ a 256) para verificar directamente

si la versión small generaliza peor que la base con datos escasos. La propuesta (ii) se implementó como demostración en el propio notebook y se describe a continuación.

5.6 Del notebook a un sistema real: inferencia con confianza y despliegue

El notebook incluye una sección de inferencia en vivo que clasifica mensajes nuevos con una medida de confianza: se comparan los logits del primer paso de generación para los tokens “ham” y “spam” mediante softmax, y las predicciones bajo un umbral (0.90 en la demostración) se marcan para revisión manual. Llevar el modelo a producción (Figura 7) implica exportar el checkpoint y publicarlo versionado (p. ej., Hugging Face Hub), envolver la inferencia en un microservicio HTTP (FastAPI) contenedorizado con Docker, calibrar el umbral según el costo asimétrico de los errores (en filtrado de SMS, bloquear un mensaje legítimo suele costar más que dejar pasar un spam) y monitorear en producción la tasa de baja confianza y los reportes de usuarios. Ese ciclo de reentrenamiento periódico con ejemplos recientes es precisamente donde la fortaleza few-shot de esta familia de modelos, documentada en [1], se vuelve una ventaja operativa: bastan pocos ejemplos nuevos para actualizar el filtro frente al drift. La latencia medida (≈ 4 ms por mensaje en GPU; el modelo es lo bastante pequeño para servirse en CPU con decenas de milisegundos) hace viable la operación en línea.

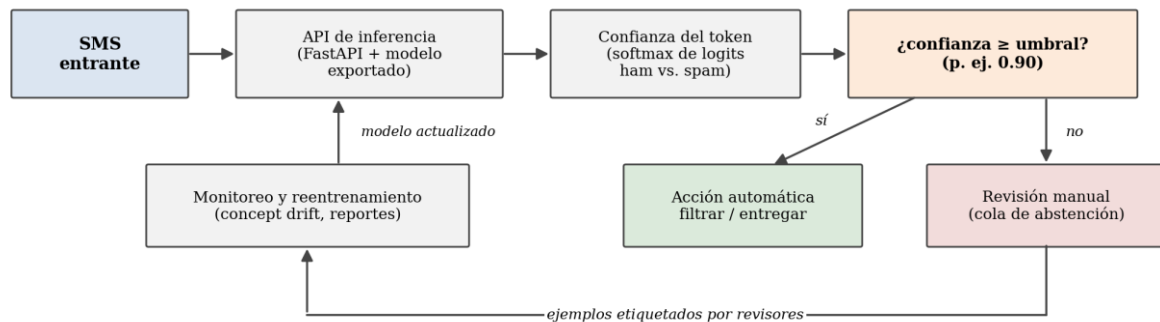


Figura 7. Proceso de despliegue del modelo como servicio de filtrado con umbral de abstención y ciclo de reentrenamiento.

6. Conclusiones

El proyecto cumplió su objetivo central: cuantificar el efecto de adaptar un modelo pre-entrenado a un problema concreto. El fine-tuning de FLAN-T5-small sobre SMS Spam Collection elevó el F1 de la clase spam de 0.229 a 0.990 y la exactitud de 0.164 a 0.997 sobre el mismo conjunto de prueba, dejando dos errores en 774 mensajes. Los resultados confirman la conclusión del paper de referencia (la superioridad de los modelos de lenguaje adaptados frente al modelo sin adaptar) y matizan una de sus decisiones: la versión small, descartada por los autores en el contexto few-shot, resulta plenamente competitiva cuando se dispone del conjunto de entrenamiento completo.

Del proceso se extraen aprendizajes transferibles: la inestabilidad numérica de T5 con float16 y su solución; que una pérdida de entrenamiento baja no garantiza capacidad de clasificación; que una métrica aislada puede enmascarar un sesgo severo en problemas desbalanceados; y que los detalles de compatibilidad de las librerías forman parte real del trabajo de ingeniería. Como trabajo futuro quedan la evaluación few-shot, la validación multi-dominio y la calibración formal del mecanismo de abstención propuesto.

Declaración de uso de inteligencia artificial

Durante el desarrollo se utilizó un asistente de inteligencia artificial (Claude, Anthropic) como apoyo en la depuración y revisión del código del notebook, en el diagnóstico de errores técnicos (inestabilidad con fp16, cambios de API de la librería transformers) y en la organización y redacción asistida del notebook y de este informe. La selección del paper, la ejecución de los experimentos, la interpretación de los resultados y las conclusiones fueron realizadas, discutidas y validadas por los integrantes del grupo, quienes asumen la responsabilidad académica del contenido completo.

Referencias

- [1] M. Labonne y S. Moran, “Spam-T5: Benchmarking Large Language Models for Few-Shot Email Spam Detection”, arXiv:2304.01238, 2023.
- [2] H. W. Chung et al., “Scaling Instruction-Finetuned Language Models”, arXiv:2210.11416, 2022.
- [3] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”, Journal of Machine Learning Research, vol. 21, n.º 140, pp. 1-67, 2020.
- [4] A. Vaswani et al., “Attention Is All You Need”, en Advances in Neural Information Processing Systems, vol. 30, 2017.
- [5] T. A. Almeida, J. M. Gómez Hidalgo y A. Yamakami, “Contributions to the Study of SMS Spam Filtering: New Collection and Results”, en Proc. 11th ACM Symposium on Document Engineering (DocEng), 2011, pp. 259-262.
- [6] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing”, en Proc. EMNLP: System Demonstrations, 2020, pp. 38-45.